Assembly

* Input output

Prev. class

- Model small
- Stack 100H → It reserves 100h bytes for stack
- Data

; The variable that are declared

```
Num DB 48
Char DB 'A'
MSG DB "My name is Rahat $"
```

- Code

```
;code segment
Main PROC ; int main
  ; code statement
  MOV AX, @Data
  MOV DS, AX   → Data segment এর সাথে যুক্ত করলাম।

  MOV AH, 2
  MOV DL, Num
  int 21h
  ; ——— New line needed here
  MOV DL, Char
  int 21h
  ; New line
  MOV AH, 9 ; string print
  LEA DX, MSG
  int 21h
  MOV AH, 4Ch
  int 21h
  Main Endp
End main ; Return 0
```

```
MOV AH, 2
MOV DL, 0AH ; mov dl, 10
int 21h
MOV DL, 0DH ; mov dl, 13
int 21h
```

LEA → load effective area.

P2

User input & output

- model small ; small, medium, compact, large, huge.
- stack 100H
- Data

- code

```
main PROC

mov AH,1        ; input call
int 21h
; input automatically stored in AL (Accumulator lower)

mov AH,2
mov DL,AL       ; print করতে হলে প্রথমে DL এ নিয়ে যেতে হয়।
int 21h

mov AH, 4CH
int 21h
main EndP

End main
```

ADD & SUB in assembly.

Assembly [illegible]
Instruction Destination, Source

- model small
- Stack 100h
- Data

```
A DB ?
B DB ?
```

- Code

```
main proc

    mov AH, 1
    int 21h
    mov A, AL
    SUB A, 48
    int 21h
    mov B, AL
    sub B, 48 ;   B = B-48

    mov BH, A
    ADD BH, B ;   BH = BH+B
    ADD BH, 48
    mov AH, 2
    mov DL, BH
    int 21h

    mov AH, 4ch
    int 21h
main Endp

End main
```

→ User input दो A को मिलाओ

→ A में 48 घटाकर संख्या निकाल लीजिए।

→ user input दो B को मिलाओ

→ New line

→ New line

```
mov DL, 0Ah
int 21h
mov DL, 0Dh
int 21h
```

P4 Built in console : print, printn

- INCLUDE 'EMU8086.INC' — header
- model small
- stack 100h
- data

- code

```
main PROC

print = print A String without new line
printN = print A String with new line

PRINT "Hello World"
printN
PRINT " BYE WORLD"


MOV AH,4CH
int 21h
main Endp

End main
```

Conditional Jmp & unconditional Jmp.

↓

compare করে দেখবে true নাকি false

↓

↓

START:
PRINTN "HERE"
JMP START

infinity loop

switch case
if case
works goto function in c.

INC BX ; BX = BX+1 ; BX++
DEC BX ; BX = BX-1 ; BX--

MOV CX, 5 ; Given
MOV BX, 0 ; initialize

for(int I=0; I<5; I++)

START:
CMP BX, CX ; compare করে দেখবে BX, CX এর ছোট বড়/ছোট/সমান সমান কিনা।
JE LAST
PRINTN "HERE"
~~JNE START~~ INC BX
JMP START

LAST:
PRINTN "The program is Ended"

Keywords

JE → Jump Equal
JNE → Jump Not equal
JG → Jump Greater
JGE → Jump Greater/equal
JL → Jump less
JLE → Jump less/equal
JA → Jump above
JB → Jump below
JC → jump if carry
JNC → Jump if no carry
JO → Jump if overflow
JNO → Jump if no overflow
⋮

***
CMP → compare

Include "Emu8086.INC"
.Model small
.stack 100H
.data
.code
Main proc

Mov AH, 4CH
INT 21H
Main Endp
End main

H/W

•) Write an assembly program that uses a loop to count down from 10 to 1 and displays each number.

•) Write a program that compares two numbers stored in memory and jumps to display the larger one.

If else & Loop in Assembly Language

- Find out if the number is negative or positive

```
Include 'Emu8086.INC'
.model small
.stack 100H
.Data
.code
main proc
  Mov BX, -10 ; RandomNumber
  CMP BX, 0 ;
  JL IF
  JGE Else

  IF:
  PrintN " The number is Negative"

  JMP END_IF
  Else:
  PrintN " The Number is positive"

  JMP END_IF

  END_IF:

   Mov AH, 4ch
   Int 21h
   Main Endp

End main.
```

GPA calculator

H.W

```
MOV BX, 10
CMP BX, 100
JGE IF1
CMP BX, 80
JGE IF2
 !
CMP BX, 20
JGE IF5

JMP ELSE

IF1:
Printn " A+ "
JMP END_IF
 !
 :
Else:
Printn " ~~Ther~~ Fail"
JMP END_IF

END_IF:
Mov AH, 4ch
 :
```

Loop Keyword

```
for(int i=0; i<n ; i++)
for (int i=N; i≥0 ; i--)* decending order
```

(কমতকরে)

```
MOV CX, 10
Top:
Printn " Loop is Running"
Loop Top ; i--; i≥0
Printn " Loop is ended".
```

P: 1+2+3+4 - - - - + N

H.W

*Assembly

- Syntax structure
- Basic instructions (keyword)
- P_1 → input, output single character.
- New line, Print function.
- P_2 → ADD, SUB
- JMP, CMP function.
- If else statement
- Loop
  - countdown from 9 to 1.
  - Finding the Maximum of two number.
  - Sum of First N Number.

Multiply Two Numbers with Dynamic initialization

```
.model small
.stack 100h
.data
a db ?
b db ?
.code
Main proc
    mov ax, @data
    mov ds, ax
    mov ah, 1
    int 21h
    Sub al, 48
    mov a, al

    mov ah, 1
    int 21h
    sub al, 48

    mul a ; al=al*a
    aam
    mov bx, ax
    mov ah, 2
    mov dl, bh
    add dl, 48
    int 21h
    mov ah, 2
    mov dl, bl
    add dl, 48
    int 21h

exit:
    mov ah, 4ch
    int 21h
main endp
end main.
```

} data initialization

aam
↳ adjust the AL after multiplication.

Lab Test
Next Sunday

Experiment Name: Demonstrate seven segment LED display and generate counting from 0 to 99 on 8051 microcontroller using an embedded C program.

Software:

① Keil µvision5

IDE. used for developing embedded applications, such as C/C++ compiler, linked/locator, macro assembler and HEX file generator.

আমরা 8051 microcontroller কে keil software এর মাধ্যমে program করে microcontroller কে instruction প্রদানের মাধ্যমে কাজ করব।

Keil এর কাজ

- Select AT89C51
- Source Group >> New file
- Write code/instruction
- Save it with .c extension.
- Right click on Source Group 1 → Add Existing file to group → Select the program.
- Select Target 1 → option for target 1 → Xtal (MHz) 11.0592 (Crystal oscillator (low cost components provides clock pulses to microprocessor controller)
- Go output → Create HEX file.
- Right click on target 1 → Build Target. >> NO Error.

A Hex file is a text file that stores machine language code in hexadecimal form. used for µcontroller, ROM.] Hex (Intel format) ASCII Text file.

Proteus

PCB Design and circuit simulator software. (paid)

Printed Circuit Board.

- AT89C51
- 7seg → 7SEG-COM-CAT-GRN
- TERMINAL → GROUND
- power → $\overline{PSEN}$ → program Store Enable.
- Click 89C51 → Select program file → go to object folder → select hex file.
- Select frequency → 11.0592